SWIGGY SALES ANALYSIS USING SQL SERVER

Project Report

Project Title : Swiggy Sales Analysis

Tool Used : Microsoft SQL Server (SQL Server Management Studio)

Prepared By : Giridharan S

Date : 28-June-2026

1. Introduction / Project Overview

The food delivery industry has experienced significant growth over the past decade, generating large volumes of transactional data every day. Companies like Swiggy process thousands of customer orders, restaurant details, food categories, locations, ratings, and pricing information. Analyzing this data helps businesses understand customer preferences, restaurant performance, pricing trends, and overall sales performance.

In this project, a **Data Warehouse** was designed using **Microsoft SQL Server** by implementing the **Star Schema** architecture. The raw Swiggy dataset was transformed into a structured database consisting of **Dimension Tables** and a **Fact Table**, making the data suitable for reporting and business analysis.

The project follows the complete **ETL (Extract, Transform, Load)** process. Initially, the raw dataset was cleaned by removing duplicate records and handling invalid data. The cleaned data was then loaded into dimension tables such as **Date**, **Location**, **Restaurant**, **Category**, and **Dish**. Finally, a centralized **Fact Table** was created by connecting all dimension tables using **Primary Keys** and **Foreign Keys**.

Using SQL queries, various business questions such as top-performing restaurants, revenue generation, customer ratings, and category-wise sales performance were analyzed to derive meaningful insights.

Background

Food delivery platforms generate millions of transactions daily. However, storing all information in a single table creates data redundancy, increases storage requirements, and slows down analytical queries.

To overcome these challenges, organizations implement **Data Warehousing** techniques such as the **Star Schema**, where descriptive information is stored separately in dimension tables while transactional information is stored in a centralized fact table. This approach improves query performance, minimizes redundancy, and enables faster business reporting.

Purpose of the Project

The primary purpose of this project is to design a structured SQL Server Data Warehouse for Swiggy sales data and perform business analysis using SQL.

The project demonstrates how raw transactional data can be transformed into an optimized database model suitable for reporting and decision-making. It also showcases SQL concepts such as **Data Cleaning**, **Normalization**, **Primary Keys**, **Foreign Keys**, **Joins**, **Aggregate Functions**, **CTEs**, **Window Functions**, and **ETL**.

Business Problem

The original Swiggy dataset stores all information in a single table, which leads to several challenges:

- High data redundancy due to repeated restaurant, location, and category information.
- Increased storage usage because identical text values are stored multiple times.
- Poor query performance when analyzing large datasets.
- Difficulty in maintaining data consistency.
- Complex reporting and business analysis.

To solve these problems, the project converts the raw dataset into a **Star Schema Data Warehouse**, where repeated information is stored only once in dimension tables and referenced through unique IDs in the fact table.

Solution Implemented

The following solution was implemented:

- Removed duplicate records using **Common Table Expressions (CTE)** and **ROW_NUMBER()**.
- Designed a **Star Schema** consisting of five Dimension Tables and one Fact Table.
- Created **Primary Key** and **Foreign Key** relationships between tables.
- Loaded data into dimension tables using SQL ETL queries.

- Populated the fact table by mapping business values to surrogate keys.
- Validated the ETL process using SQL JOIN operations.
- Performed analytical SQL queries to generate business insights.

Project Workflow

Raw Swiggy Dataset

⬇

Data Cleaning

(Duplicate Removal using CTE)

⬇

Create Dimension Tables

⬇

Create Fact Table

⬇

Load Dimension Tables (ETL)

⬇

Load Fact Table (ETL)

⬇

Data Validation

⬇

Business Analysis Queries

⬇

Business Insights

2. Objectives of the Project

The primary objectives of this project are:

- To design a Data Warehouse using the Star Schema architecture.
- To clean the raw Swiggy dataset by removing duplicate records.
- To create Dimension Tables for Date, Location, Restaurant, Category, and Dish.
- To create a centralized Fact Table for storing transactional order data.
- To establish relationships using Primary Keys and Foreign Keys.
- To implement the ETL (Extract, Transform, Load) process using SQL Server.
- To validate the data using SQL JOIN operations.
- To perform analytical SQL queries for business reporting.
- To identify top-performing restaurants based on orders and revenue.
- To analyze food categories, customer ratings, pricing trends, and location-wise sales performance.
- To improve SQL skills in Data Warehousing, ETL, and Business Intelligence.

3. Tools & Technologies Used

Tool / Technology	Purpose
Microsoft SQL Server	Database Management System
SQL Server Management Studio (SSMS)	SQL Development Environment
SQL (T-SQL)	Query Writing and Data Analysis
Star Schema	Data Warehouse Design
ETL	Data Extraction, Transformation and Loading
GitHub	Version Control and Project Documentation

4. Skills Demonstrated

During this project, the following technical skills were applied:

- SQL Query Writing
- Data Cleaning
- Data Warehousing

- Star Schema Design
- Fact and Dimension Table Design
- Primary Key & Foreign Key Implementation
- Common Table Expressions (CTE)
- Window Functions
- INNER JOIN Operations
- Aggregate Functions
- ETL Process
- Business Data Analysis
- Database Normalization
- Data Validation

5. Dataset Description

Dataset Name

Swiggy Food Sales Dataset

Source

Kaggle Public Dataset (Swiggy Food Sales Dataset)

Dataset Overview

The Swiggy dataset contains food order transaction details collected from different restaurants across various cities and states. The dataset includes information such as order date, restaurant name, food category, dish name, price, customer rating, and rating count.

The dataset is used to build a **SQL Server Data Warehouse** for performing business analysis and generating meaningful insights using SQL queries.

Dataset Structure

Column Name	Description
State	State where the order was placed
City	City where the restaurant is located
Order_Date	Date of the order
Restaurant_Name	Name of the restaurant
Location	Area/Locality of the restaurant

Category	Food category (South Indian, Chinese, etc.)
Dish_Name	Name of the dish ordered
Price_INR	Price of the dish in Indian Rupees
Rating	Customer rating for the dish/restaurant
Rating_Count	Number of customer ratings

Business Importance of the Dataset

The dataset helps answer several business questions such as:

- Which restaurant receives the highest number of orders?
- Which food category generates the highest revenue?
- Which city contributes the highest sales?
- Which restaurants have the highest customer ratings?
- Which dishes are most popular?
- How does revenue vary across different locations?

These insights help businesses improve pricing strategies, restaurant partnerships, and customer satisfaction.

6. Data Cleaning

Introduction

Raw datasets often contain duplicate records, missing values, and inconsistent data. Before performing any analysis, the data must be cleaned to ensure accuracy and reliability.

In this project, the dataset was cleaned before loading it into the Data Warehouse.

Data Cleaning Steps Performed

The following data preparation steps were carried out:

- Checked the dataset for duplicate records.
- Removed duplicate rows using **CTE (Common Table Expression)** and **ROW_NUMBER()**.
- Verified that the **Order_Date** column did not contain NULL values before loading the Date Dimension.

- Extracted only unique values while loading Dimension Tables using the **DISTINCT** keyword.
- Maintained referential integrity by loading only valid records into the Fact Table.

Duplicate Removal

Duplicate records increase storage usage and produce incorrect analytical results.

To eliminate duplicate records, a **Common Table Expression (CTE)** along with the **ROW_NUMBER()** window function was used.

SQL Query

```
--Delete Duplicates

WITH CTE AS (
SELECT *, ROW_NUMBER() OVER (
	PARTITION BY State, City, Order_date, Restaurant_Name, Location,
Category, Dish_Name, Price_INR, Rating, Rating_Count
	ORDER BY (SELECT NULL)
	) AS rn
	FROM Swiggy_data
	)
	DELETE FROM CTE WHERE rn>1
```

Explanation

The query groups identical records using the **PARTITION BY** clause. The **ROW_NUMBER()** function assigns a sequence number to each row within the same group.

- The first occurrence receives **Row Number = 1**
- Duplicate rows receive **Row Number = 2, 3, 4...**

Finally, all rows where **Row Number > 1** are deleted, leaving only unique records in the dataset.

Example

Before Cleaning

Restaurant	Dish	Price
A2B	Dosa	80
A2B	Dosa	80
A2B	Dosa	80

ROW_NUMBER()

Restaurant	Dish	Price	Row Number
A2B	Dosa	80	1
A2B	Dosa	80	2
A2B	Dosa	80	3

After Cleaning

Restaurant	Dish	Price
A2B	Dosa	80

SQL Concepts Used

Common Table Expression (CTE)

Definition

A Common Table Expression (CTE) is a temporary result set that simplifies complex SQL queries and improves readability.

Purpose in this Project

The CTE was used to temporarily identify duplicate records before deleting them.

ROW_NUMBER()

Definition

ROW_NUMBER() is a window function that assigns a unique sequential number to each row within a partition.

Purpose in this Project

It identifies duplicate records by assigning row numbers to rows having identical values.

PARTITION BY

Definition

PARTITION BY divides the dataset into groups based on selected columns before applying the window function.

Purpose in this Project

Rows with the same State, City, Restaurant, Dish, Price, Rating, and Order Date were grouped together.

OVER() Clause

Definition

The OVER() clause defines how the window function should operate.

Purpose in this Project

It specifies the grouping (PARTITION BY) and ordering rules for ROW_NUMBER().

DISTINCT

Definition

The DISTINCT keyword returns only unique values by removing duplicates from the result set.

Purpose in this Project

DISTINCT was used while loading Dimension Tables so that repeated values such as restaurants, locations, dishes, and categories were inserted only once.

7. Database Design

Introduction

To improve data organization and query performance, the raw Swiggy dataset was converted into a **Star Schema Data Warehouse**.

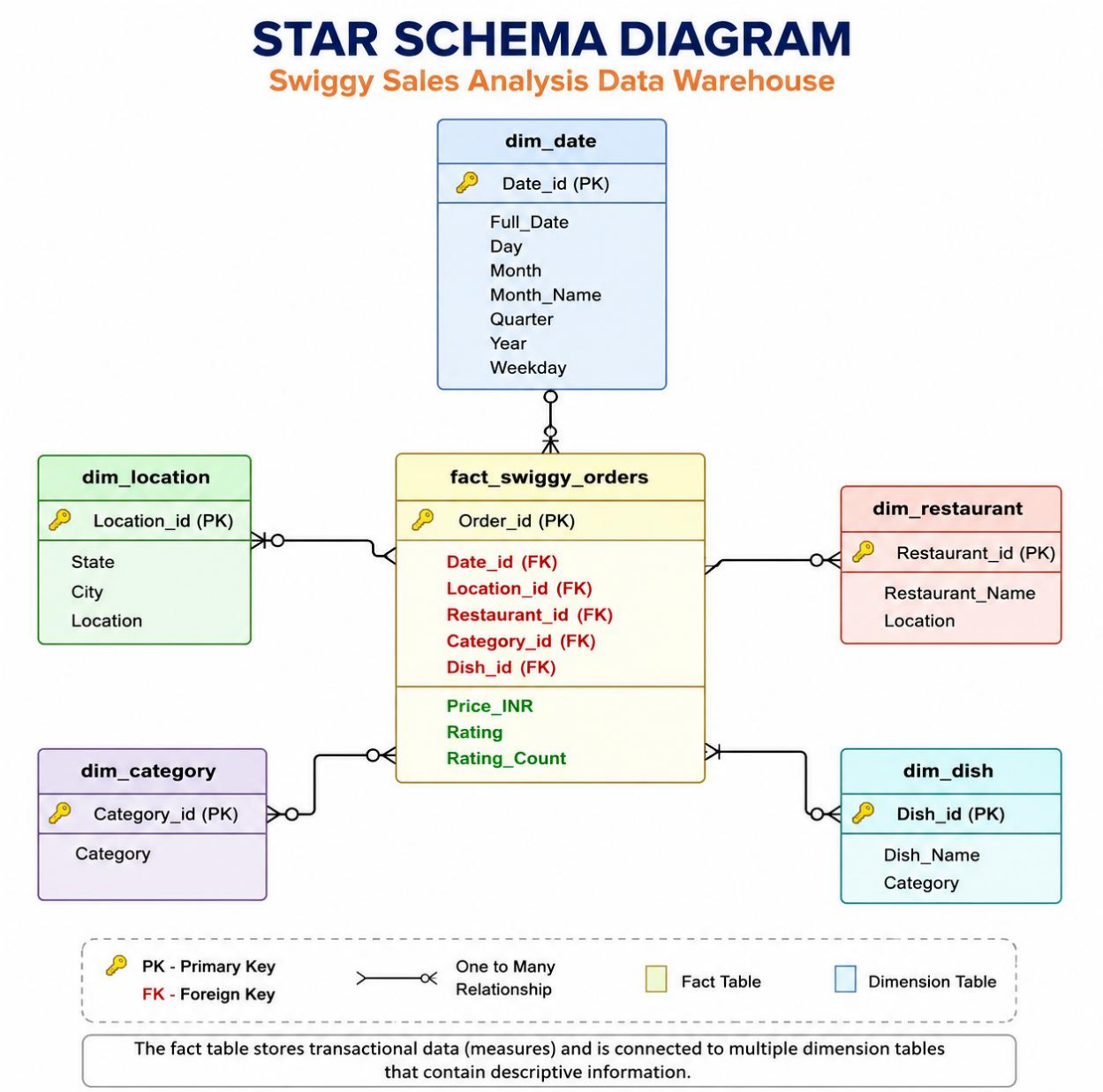
Instead of storing all information in a single table, descriptive data was separated into Dimension Tables, while transactional data was stored in a centralized Fact Table.

This design reduces data redundancy, improves query performance, and simplifies business reporting.

Advantages of Star Schema

- Reduces data redundancy.
- Improves query execution speed.
- Simplifies analytical reporting.
- Easy to maintain and extend.
- Supports Business Intelligence and Data Warehousing applications.
- Makes SQL JOIN operations faster and more efficient.

Star Schema Architecture



8. Dimension Tables

Introduction

Dimension Tables store descriptive or categorical information about business entities. They provide context to the numerical data stored in the Fact Table. In this project, five dimension tables were created to eliminate redundancy and improve query performance.

The Dimension Tables created are:

- dim_date
- dim_location
- dim_restaurant
- dim_category

- dim_dish

8.1 Date Dimension (dim_date)

Purpose

The Date Dimension stores all date-related information such as year, month, quarter, week, and day. Instead of storing these values repeatedly for every order, they are stored once and referenced using a unique **Date_id**.

Table Structure

Column Name	Description
Date_id	Primary Key
Full_Date	Order Date
Year	Year of Order
Month	Month Number
Month_Name	Month Name
Quarter	Quarter of the Year
Day	Day of Month
Week	Week Number

Benefits

- Eliminates repeated date information.
- Enables time-based analysis.
- Simplifies monthly, quarterly, and yearly reporting.

8.2 Location Dimension (dim_location)

Purpose

The Location Dimension stores geographical information related to each order. It contains the state, city, and locality where the restaurant is located.

Table Structure

Column Name	Description
Location_id	Primary Key
State	State Name
City	City Name

Location	Restaurant Area/Locality

Benefits

- Removes repeated location values.
- Enables city-wise and state-wise sales analysis.
- Improves reporting efficiency.

8.3 Restaurant Dimension (dim_restaurant)

Purpose

The Restaurant Dimension stores unique restaurant names. Each restaurant is assigned a unique Restaurant_id, which is referenced in the Fact Table.

Table Structure

Column Name	Description
Restaurant_id	Primary Key
Restaurant_Name	Restaurant Name

Benefits

- Eliminates duplicate restaurant names.
- Simplifies restaurant performance analysis.
- Reduces storage requirements.

8.4 Category Dimension (dim_category)

Purpose

The Category Dimension stores different food categories available in the dataset.

Table Structure

Column Name	Description
Category_id	Primary Key
Category	Food Category

Benefits

- Prevents repeated category values.
- Enables category-wise revenue analysis.
- Supports food preference analysis.

8.5 Dish Dimension (dim_dish)

Purpose

The Dish Dimension stores all unique dish names available in the dataset.

Table Structure

Column Name	Description
Dish_id	Primary Key
Dish_Name	Dish Name

Benefits

- Removes duplicate dish names.
- Enables dish popularity analysis.
- Simplifies menu-based reporting.

9. Fact Table

Introduction

The Fact Table is the central table in the Star Schema. It stores transactional data (measures) and references all Dimension Tables using Foreign Keys.

In this project, the Fact Table is **fact_swiggy_orders**.

Fact Table Structure

Column Name	Description
Order_id	Primary Key
Date_id	Foreign Key
Price_INR	Order Price
Rating	Customer Rating
Rating_Count	Number of Ratings
Location_id	Foreign Key
Restaurant_id	Foreign Key
Category_id	Foreign Key
Dish_id	Foreign Key

Measures

The following are the numerical measures stored in the Fact Table:

- Price_INR
- Rating
- Rating_Count

These values are used for aggregation functions such as:

- SUM()
- AVG()
- COUNT()
- MAX()
- MIN()

Relationships

The Fact Table is connected to all Dimension Tables through Foreign Keys.

Fact Table Column	References
Date_id	dim_date
Location_id	dim_location
Restaurant_id	dim_restaurant
Category_id	dim_category
Dish_id	dim_dish

10. Primary Key and Foreign Key

Primary Key

A Primary Key is a column that uniquely identifies each record in a table.

Characteristics

- Unique value
- Cannot contain NULL values
- Only one Primary Key per table

Example

dim_restaurant

Restaurant_id	Restaurant_Name

1	A2B
2	KFC
3	Domino's

Here, **Restaurant_id** is the Primary Key because every restaurant has a unique ID.

Foreign Key

A Foreign Key is a column that establishes a relationship between two tables by referencing the Primary Key of another table.

Example

Fact Table

Restaurant_id	Price
2	250

Here, Restaurant_id = 2 refers to **KFC** in the Restaurant Dimension.

Thus,

fact_swiggy_orders.Restaurant_id

↓

dim_restaurant.Restaurant_id

Advantages of Primary & Foreign Keys

- Maintain data integrity.
- Prevent invalid data entry.
- Establish relationships between tables.
- Improve consistency.
- Support efficient JOIN operations.

11. ETL (Extract, Transform, Load)

Introduction

ETL stands for **Extract, Transform, and Load**. It is a process used in Data Warehousing to collect data from source systems, clean and transform it, and load it into a structured database for analysis.

In this project, ETL was implemented using SQL Server.

Extract

The raw Swiggy dataset was imported into SQL Server as the source table (**Swiggy_data**).

Transform

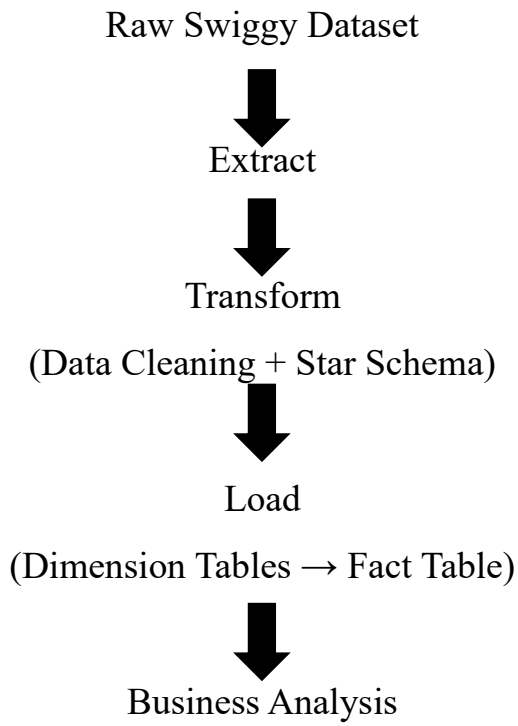
During transformation, the following operations were performed:

- Removed duplicate records.
- Checked for NULL values.
- Extracted unique values using DISTINCT.
- Created Dimension Tables.
- Generated surrogate keys using IDENTITY.
- Established relationships using Primary Keys and Foreign Keys.

Load

- Dimension Tables were populated.
- Fact Table was populated using JOIN operations.
- Data was validated after loading.

ETL Workflow



12. Loading Dimension Tables

The Dimension Tables were loaded using the **INSERT INTO ... SELECT DISTINCT** statement.

Using **DISTINCT** ensured that only unique values were inserted into each dimension table.

For example:

- Unique Restaurants → dim_restaurant
- Unique Categories → dim_category
- Unique Dishes → dim_dish
- Unique Locations → dim_location
- Unique Dates → dim_date

This reduced redundancy and improved database normalization.

13. Loading Fact Table

After loading all Dimension Tables, the Fact Table was populated.

The Fact Table does not store text values such as restaurant names or category names directly. Instead, it stores the corresponding IDs obtained from the Dimension Tables.

The data was loaded using multiple **INNER JOIN** operations between the source table and Dimension Tables.

This process mapped business values such as **Order_Date**, **Restaurant_Name**, **Category**, **Dish_Name**, and **Location** to their corresponding surrogate keys before inserting them into the Fact Table.

Benefits of this Approach

- Reduced storage space.
- Faster query execution.
- Better scalability.
- Improved data consistency.
- Easy integration with Business Intelligence tools such as Power BI.

14. Data Validation

Introduction

After completing the ETL process, it is essential to verify that all records have been correctly loaded into the Data Warehouse. Data Validation ensures that the Fact Table contains valid Foreign Key references to all Dimension Tables and that the relationships between tables are accurate.

In this project, **INNER JOIN** operations were used to validate the loaded data by combining the Fact Table with all Dimension Tables.

Validation Query

```sql
SELECT * FROM fact_swiggy_orders f
JOIN dim_date d ON f.date_id = d.date_id
JOIN dim_location l ON f.Location_id = l.Location_id
JOIN dim_restaurant r ON f.Restaurant_id = r.Restaurant_id
JOIN dim_category c ON f.Category_id = c.Category_id
JOIN dim_dish di ON f.Dish_id = di.Dish_id;
```

Explanation

The validation query joins the Fact Table with all Dimension Tables using their respective Foreign Keys.

This confirms that:

- Every **Date_id** correctly maps to a valid date.
- Every **Location_id** maps to the correct state, city, and location.
- Every **Restaurant_id** maps to the correct restaurant.
- Every **Category_id** maps to the appropriate food category.
- Every **Dish_id** maps to the correct dish.

Successful execution of this query indicates that the ETL process has correctly populated the Data Warehouse.

Benefits of Data Validation

- Verifies ETL accuracy.
- Ensures referential integrity.
- Detects missing or incorrect mappings.
- Confirms that all relationships are correctly established.
- Prevents incorrect business reports.

15. Business Analysis Queries

After validating the data, several analytical SQL queries were executed to generate business insights.

Query 1 – Top 10 Restaurants by Orders

Objective:

To identify the restaurants receiving the highest number of customer orders.

Business Value:

Helps identify the most popular restaurants and supports partnership and promotional decisions.

Query 2 – Top 10 Restaurants by Revenue

Objective:

To calculate total revenue generated by each restaurant.

Business Value:

Identifies high-performing restaurants contributing the highest sales revenue.

Query 3 – Revenue by Food Category

Objective:

To analyze revenue generated by different food categories.

Business Value:

Helps understand customer food preferences and optimize menu offerings.

Query 4 – Revenue by State

Objective:

To compare revenue across different states.

Business Value:

Supports regional sales analysis and expansion planning.

Query 5 – Revenue by City

Objective:

To identify cities generating the highest revenue.

Business Value:

Helps allocate marketing resources effectively.

Query 6 – Highest Rated Restaurants

Objective:

To identify restaurants with the best customer ratings.

Business Value:

Measures customer satisfaction and restaurant quality.

Query 7 – Most Popular Dishes

Objective:

To identify dishes with the highest number of orders.

Business Value:

Supports menu optimization and inventory planning.

Query 8 – Average Rating by Restaurant

Objective:

To calculate the average customer rating for each restaurant.

Business Value:

Provides insights into customer satisfaction and service quality.

16. Key Business Insights

Based on the SQL analysis, the following insights can be obtained:

- The top-performing restaurants generate the highest number of customer orders and revenue.
- Certain food categories contribute significantly to overall sales.
- Revenue varies across different cities and states.
- Customer ratings indicate the quality of restaurants and their services.
- Popular dishes represent customer preferences and can be prioritized in promotions.

- Geographic analysis helps identify high-performing business locations.

These insights support better business decision-making and strategic planning.

17. Challenges Faced

During the development of this project, several challenges were encountered:

- Duplicate records in the source dataset.
- Designing an efficient Star Schema.
- Mapping business values to surrogate keys.
- Establishing Primary Key and Foreign Key relationships.
- Loading Fact Table data without violating referential integrity.
- Validating the ETL process.

These challenges were resolved using SQL concepts such as CTE, ROW_NUMBER(), DISTINCT, INNER JOIN, and Foreign Key constraints.

18. Conclusion

This project successfully demonstrates the design and implementation of a SQL Server Data Warehouse for Swiggy sales data using the Star Schema architecture.

The raw dataset was cleaned and transformed into a structured database consisting of Dimension Tables and a centralized Fact Table. Primary Key and Foreign Key relationships were established to maintain data integrity and support efficient querying.

The ETL process ensured accurate loading of data into the warehouse, while SQL validation queries confirmed the correctness of the relationships. Finally, analytical SQL queries were executed to generate meaningful business insights regarding restaurant performance, revenue, customer ratings, food categories, and geographic sales trends.

Overall, this project strengthened practical knowledge of SQL, Data Warehousing, ETL, Database Design, and Business Intelligence concepts.

19. Future Enhancements

The project can be further enhanced by implementing the following features:

- Develop an interactive Power BI dashboard connected to the SQL Server database.
- Automate the ETL process using SQL Server Integration Services (SSIS).
- Create Stored Procedures for reusable analytical queries.
- Implement Indexing to improve query performance.

- Schedule automatic data refreshes.
- Integrate additional datasets such as customer demographics and delivery partner information.
- Build predictive models for demand forecasting using Machine Learning.

20. References

1. Microsoft SQL Server Documentation
   https://learn.microsoft.com/sql/
2. SQL Server Management Studio (SSMS) Documentation
   https://learn.microsoft.com/sql/ssms/
3. Kaggle – Swiggy Food Sales Dataset
4. W3Schools SQL Tutorial
   https://www.w3schools.com/sql/
5. SQLBolt – Interactive SQL Learning
   https://sqlbolt.com/

Appendix

The following SQL scripts were developed as part of this project:

- Data Cleaning
- Create Dimension Tables
- Create Fact Table
- Load Dimension Tables
- Load Fact Table
- Data Validation
- Business Analysis Queries

THANK YOU